

CRM Features · Documentación técnica completa

FastChat DJ · Plataforma WhatsApp CRM con IA

Generado automáticamente desde la fuente `.md`
Versión actualizada · Abril 2026

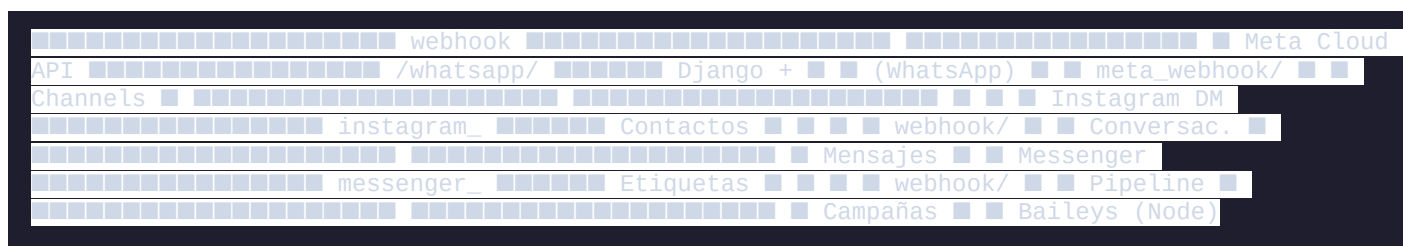
CRM Features · Guía completa

Qué cubre este documento. Todas las funcionalidades CRM que se añadieron al núcleo WhatsApp/IA para acercar FastChat DJ al nivel de plataformas como WATI, Respond.io o Kommo, y para permitir **campañas de Instagram / Facebook Ads que terminan en WhatsApp** con atribución completa.

Índice

1. [Arquitectura general](#)
2. [Puesta en marcha \(primera vez\)](#)
3. [Etiquetas \(tags\)](#)
4. [Pipeline de ventas \(Kanban\)](#)
5. [Campañas masivas](#)
6. [Horarios de atención](#)
7. [Round-robin: asignación automática a agentes](#)
8. [Click-to-WhatsApp \(CTWA\) / Click-to-Instagram — atribución Meta Ads](#)
9. [Meta Conversions API \(CAPI\)](#)
10. [Instagram DM + Facebook Messenger](#)
11. [Analytics dashboard](#)
12. [API REST v1](#)
13. [Webhooks salientes \(estilo Zapier\)](#)
14. [Cron jobs](#)
15. [Flujo end-to-end: anuncio IG → venta cerrada → Purchase a Meta Ads](#)
16. [Tutorial paso a paso \(demo\)](#)

Arquitectura general



```
webhook_ handler/ Meta CAPI
(Lead/Purchase) conversiones salientes
Webhooks salientes
integraciones
```

Modelos nuevos (todos en `whatsapp/models.py`):

Modelo	Propósito
<code>EtiquetaContacto</code>	Tags libres para segmentar contactos.
<code>PipelineVenta</code> , <code>EtapaPipeline</code> , <code>ConversacionEnPipeline</code> , <code>HistorialEtapaPipeline</code>	Tablero Kanban de ventas.
<code>HorarioAtencion</code> , <code>ExcepcionHorario</code>	Business hours + feriados.
<code>Campana</code> , <code>EnvioCampana</code>	Broadcasts segmentados.
<code>PixelMeta</code> , <code>EventoCAPI</code>	Meta Conversions API.
<code>ConfigInstagram</code> , <code>ConfigMessenger</code>	Credenciales IG DM + Messenger.
<code>DisponibilidadAgente</code> , <code>AsignacionAutomatica</code>	Round-robin.
<code>WebhookSaliente</code> , <code>EntregaWebhookSaliente</code>	Integraciones outbound.

Campos agregados a modelos existentes:

- `ConversacionWhatsApp`: `origen_canal`, `ctwa_clid`, `ad_id`, `adset_id`, `campaign_id`, `referral_*`, `capi_lead_enviado`, `capi_purchase_enviado`, `campana_origen`.
- `Contacto`: `etiquetas` (M2M), `canal`, `external_id`.
- `SesionWhatsApp`: `mensaje_fuera_horario`, `zona_horaria`, `auto_asignar_round_robin`, `pixel_meta` (FK).
Se extendieron los `PROVEEDORES_SESSION` con `instagram` y `messenger`.

Puesta en marcha (primera vez)

Tras hacer `git pull`:

```
# 1. Aplicar migraciones nuevas python manage.py migrate whatsapp # 2. (Opcional) seed de
pipeline por defecto python manage.py shell -c "from whatsapp.models import PipelineVenta,
EtapaPipeline p, _ = PipelineVenta.objects.get_or_create(nombre='Ventas',
```

```
defaults={'es_default': True}) for i, (n, c, prob, g, l) in enumerate([ ('Prospecto', '#6c757d', 10, False, False), ('Contactado', '#0dcaf0', 25, False, False), ('Cotizando', '#ffc107', 50, False, False), ('Negociando', '#fd7e14', 75, False, False), ('Cerrado ganado', '#198754', 100, True, False), ('Perdido', '#dc3545', 0, False, True), ]):
EtapaPipeline.objects.get_or_create(pipeline=p, nombre=n, defaults={'orden': i, 'color': c, 'probabilidad_cierre': prob, 'es_ganado': g, 'es_perdido': l}) " # 3. Reiniciar el servidor
(daphne o runserver) daphne -b 0.0.0.0 -p 8000 fastchatdj.asgi:application
```

Menú lateral — tras la migración `0033_crear_modulos_crm` aparecen estos nuevos módulos (pueden asignarse a grupos desde Seguridad → Módulos):

- `/whatsapp/etiquetas/`
- `/whatsapp/pipeline/`
- `/whatsapp/campanas/`
- `/whatsapp/horarios/`
- `/whatsapp/analytics/`

Etiquetas (tags)

Qué resuelve: segmentación libre para campañas y filtros. Antes sólo existían 6 clasificaciones fijas.

URL: `/whatsapp/etiquetas/`

Flujo de uso: 1. Abre el módulo **Etiquetas**. 2. Haz clic en **Nueva etiqueta** → asigna nombre + color + descripción. 3. Desde la ficha de un contacto (o por bulk vía API), aplícalas. 4. Al crear una **Campaña** puedes filtrar "incluir / excluir" por estas etiquetas.

API:

```
POST /whatsapp/api/v1/etiquetas/aplicar/ X-API-Key: <NODE_SECRET_KEY> Content-Type: application/json {"contacto_ids": [12, 34, 56], "etiqueta_ids": [3, 7], "remover": false}
```

Pipeline de ventas (Kanban)

Qué resuelve: vista Kanban con arrastrar-y-soltar. Cada conversación puede ser una tarjeta con valor estimado y probabilidad de cierre.

URL: `/whatsapp/pipeline/`

Flujo de uso: 1. Crea un **Pipeline** (ej: "Ventas B2C"). Puede haber varios. 2. Dentro del pipeline, crea **Etapas** ordenadas (columnas). Una etapa puede marcarse como `es_ganado=True` (dispara evento Purchase a Meta CAPI automáticamente) o `es_perdido=True`. 3. Desde una conversación, agrégala como tarjeta a una etapa (acción `agregar_card`). 4. En el Kanban, arrastra tarjetas entre columnas. El sistema: - Guarda el cambio en

`ConversacionEnPipeline.etapa`. - Registra el movimiento en `HistorialEtapaPipeline` (para análisis de funnel). - Si la nueva etapa es `es_ganado=True`, dispara **Purchase** a Meta CAPI con `valor_estimado` como monto.

Forecast ponderado: el dashboard analytics calcula `valor × probabilidad/100` por etapa.

Campañas masivas

Qué resuelve: envío programado a miles de contactos con throttling, ventana horaria y segmentación por etiquetas.

URL: `/whatsapp/campanas/`

Tipos soportados: - `texto`: mensaje plano con placeholders `{nombre}` y `{numero}`. - `plantilla`: plantilla Meta aprobada (obligatoria para WhatsApp fuera de la ventana 24h). - `media`: texto + adjunto.

Flujo de uso: 1. **Crear campaña** → modal con: - Sesión emisora (WhatsApp/IG/Messenger). - Tipo (texto/plantilla/media). - Mensaje o selección de plantilla. - Etiquetas **incluir** y **excluir**. - Canales permitidos. - Throttle (`msg/min`) y ventana horaria opcional. 2. Queda en estado **borrador** con audiencia resuelta. 3. Haz clic en **Enviar ahora** (o programa fecha futura) → pasa a **programada**. 4. El cron `ejecutar_campanas.py` corre cada minuto: - Materializa la audiencia en `EnvioCampana` (uno por contacto, idempotente). - Envía hasta `throttle_por_minuto` mensajes por ciclo, respetando ventana horaria. - Registra éxito/fallo por envío. - Cuando ya no quedan pendientes, marca la campaña **completada**. 5. Desde el listado, **Detalle** muestra progreso + tabla de últimos 200 envíos.

Cron setup (crontab Linux):

```
* * * * * cd /ruta/fastchatdj && /ruta/.venv/bin/python cron_jobs/ejecutar_campanas.py
```

Windows Task Scheduler: programa el mismo comando cada minuto.

Pausar / cancelar: desde el listado, botones **Pausar** y **Cancelar**.

Horarios de atención

Qué resuelve: responder "estamos fuera de horario" automáticamente y pausar round-robin fuera de las horas configuradas.

URL: `/whatsapp/horarios/`

Flujo de uso: 1. Selecciona una sesión en el panel izquierdo. 2. Configura el **mensaje fuera de horario** y la **zona horaria** (TZ name, ej. `America/Guayaquil`). 3. Agrega **horarios semanales** — filas día/desde/hasta. Si no agregas ninguno, la sesión se considera abierta 24/7. 4. Agrega **excepciones** (feriados, días especiales). Pueden ser: - Día cerrado (motivo "Feriado") - Día abierto con horario especial (opcional).

Integración automática: el helper `whatsapp/services_horarios.py:dentro_de_horario(sesion)` se usa en `process_incoming_message` si quieres condicionar respuestas. Se expone para que lo llames desde tu lógica custom.

Round-robin: asignación automática a agentes

Qué resuelve: distribuir conversaciones nuevas entre agentes humanos sin intervención manual.

Modelo clave: `DisponibilidadAgente`.

Setup: 1. **Admin** → **Disponibilidad de agentes** → crea un registro por cada usuario agente. 2. Marca `disponible=True`, define `max_conversaciones` (tope de carga simultánea), y opcionalmente las `sesiones` y `departamentos` que atiende. 3. En **Sesiones WhatsApp**, activa la opción `auto_asignar_round_robin` en las sesiones que quieras que usen esta lógica.

Criterios de asignación (en orden): 1. Disponible (`disponible=True`). 2. No excede su `max_conversaciones`. 3. La sesión entra en su lista (si vacía, atiende todas). 4. **Menor carga** actual; empate → el que lleva **más tiempo sin asignación**.

Traza: cada asignación se registra en `AsignacionAutomatica` y también en `HistorialAsignacion`. Thread-safe via `select_for_update()`.

Disparo manual vía API:

```
POST /whatsapp/api/v1/conversaciones/123/asignar/ X-API-Key: <NODE_SECRET_KEY> Content-Type: application/json {"auto": true}
```

Click-to-WhatsApp (CTWA) / Click-to-Instagram · atribución Meta Ads

Qué resuelve: saber de qué anuncio / campaña vino cada conversación para calcular ROI real.

Cómo se captura: Meta envía un bloque `referral` en el payload del primer mensaje cuando el usuario llega desde un anuncio. Nuestro `meta_webhook_view.py` ya lo parsea y guarda en la conversación:

```
# Campos que se rellenan en ConversacionWhatsApp al llegar el primer mensaje origen_canal #
"whatsapp" | "instagram" | "messenger" referral_source_type # "AD", "POST", "PAGE", ...
ctwa_click # Click ID Meta (único por click) ad_id adset_id campaign_id referral_source_url #
URL del post/ad origen referral headline referral body referral payload json # payload
completo para reproceso
```

Dónde lo ves: - En la conversación misma (API y admin). - En el Kanban: un ícono ■ en tarjetas provenientes de CTWA. - En el dashboard Analytics, tabla **ROI por campaña CTWA**: conversaciones / leads / clientes / tasa de conversión agrupados por `campaign_id` + `ad_id`.

Meta Conversions API (CAPI)

Qué resuelve: reportar eventos **Lead** y **Purchase** de vuelta a Meta Ads para que el algoritmo pueda optimizar la pauta con conversiones offline.

Setup: 1. En Meta Business Manager crea un **Pixel/Dataset** y genera un **CAPI access token**. 2. En el admin Django → **Pixels Meta (CAPI)** → crea registro con: - `pixel_id` (ej: `1234567890`). - `access_token`. - `test_event_code` (opcional, para modo test). 3. En **Sesiones WhatsApp**, asigna el `pixel_meta` a la sesión correspondiente.

Cuándo se dispara automáticamente: - **Lead:** al crearse una conversación con `ctwa_clid` o `ad_id` presentes (ver `process_incoming_message` en `view_webhook_handler.py:~530`). - **Purchase:** al mover una tarjeta Kanban a una etapa con `es_ganado=True`. Usa `card.valor_estimado` como `value`.

Disparo manual vía API:

```
POST /whatsapp/api/v1/capi/evento/ X-API-Key: <NODE_SECRET_KEY> Content-Type: application/json
{"conversacion_id": 1234, "event_name": "Purchase", "value": 150.00, "currency": "USD"}
```

Qué envía el servicio: - `user_data` con phone/nombre hasheados (SHA-256) + `ctwa_clid`. - `custom_data` con `value`, `currency`, `ad_id`, `campaign_id`. - `action_source:` `business_messaging`, `messaging_channel:` `whatsapp|instagram|messenger`. - Si el pixel tiene `test_event_code`, el evento sale en modo test (visible solo en **Events Manager** → **Test Events**).

Auditoría: todos los eventos quedan en **EventoCAPI** con payload y respuesta de Meta.

Instagram DM + Facebook Messenger

Qué resuelve: atender mensajes de Instagram DM y Messenger con el mismo inbox y la misma IA que WhatsApp.

Setup Instagram: 1. Crea una **sesión** en `/whatsapp/sesiones/` con proveedor **Instagram DM**. 2. En el admin → **Configuraciones Instagram** → crea registro linkeado a esa sesión: - `ig_user_id` (Instagram Business Account ID). - `page_id` (Facebook Page vinculada). - `access_token` (Page token con permisos `instagram_manage_messages`). - `app_secret` (para validar HMAC del webhook). - `webhook_verify_token` (genera uno seguro, ej. `secrets.token_urlsafe(32)`). 3. En Meta Developer Portal → tu app → Instagram Graph API → Webhook: - Callback URL: `https://tu-dominio.com/whatsapp/instagram_webhook/` - Verify Token: el que pusiste arriba. - Suscribe los campos: `messages`, `messaging_postbacks`, `message_reactions`.

Setup Messenger: idéntico, pero con `ConfigMessenger`, URL `/whatsapp/messenger_webhook/` y permisos `pages_messaging`.

Send: a través del dispatcher habitual:

```
from whatsapp_services import get_whatsapp_service = get_whatsapp_service(sesion) #
devuelve InstagramService o MessengerService automáticamente
service.send_text_message(sesion.session_id, recipient_id, "Hola!")
```

Deduplicación multi-canal: cada contacto se marca con `canal` + `external_id` (PSID/IGSID/wa_id). Queda pendiente a futuro consolidar identidades cross-canal en un **ContactoUnificado**.

Analytics dashboard

URL: `/whatsapp/analytics/`

Qué muestra (todo filtrable por rango 7/30/90 días):

Bloque	Fuente
KPIs: conversaciones, abiertas, leads, clientes, mensajes IA, CAPI enviados	<code>ConversacionWhatsApp</code> , <code>Mensajewhatsapp</code> , <code>EventoCAPI</code>
Conversaciones por día (chart line)	Agrupado por <code>TruncDate(fecha_registro)</code>
Por canal de origen (doughnut)	<code>origen_canal</code>
Por clasificación (bar)	<code>clasificacion</code> 0..5
Sentimiento (pie)	<code>sentimiento</code>
Ranking agentes (horizontal bar)	<code>Mensajewhatsapp.agente</code>
ROI CTWA (tabla)	agrupado por <code>campaign_id</code> + <code>ad_id</code>
Pipeline forecast (tabla)	<code>ConversacionEnPipeline</code> × <code>EtapaPipeline.probabilidad_cierre</code>

Datos en JSON: misma URL con `?action=data` devuelve todo para consumo externo / integraciones BI.

API REST v1

Base URL: `/whatsapp/api/v1/`

Autenticación: header `X-API-Key: <NODE_SECRET_KEY>` (reusamos la llave que usa Node para hablar con Django).

Rate limit: 120 requests/min por key.

Endpoints:

Método	Ruta	Descripción
GET	<code>/contactos/</code>	Lista + filtros: <code>?sesion=<id>&canal=<>&q=<>&etiqueta=<i</code>

POST	/contactos/	Crear {sesion_id, numero, nombre?, canal?}
GET	/contactos/<id>/	Detalle
GET	/conversaciones/	Lista + filtros: ?sesion=&estado=abierta cerrada&clasif
GET	/conversaciones/<id>/mensajes/?limit=50	Historial
POST	/conversaciones/<id>/asignar/	{"usuario_id":42} o {"auto":true}
POST	/conversaciones/<id>/etapa/	{"etapa_id": 5, "valor_estimado": 150, "moneda":"USD"}
POST	/mensajes/enviar/	{"sesion_id":1, "destino":"593999...", "texto":"Hola"}
POST	/etiquetas/aplicar/	{"contacto_ids":[...], "etiqueta_ids":[...], "remove":false}
POST	/capi/evento/	{"conversacion_id":1, "event_name":"Purchase", "value":150}
GET	/campanas/<id>/stats/	Stats con breakdown por estado

Ejemplo curl:

```
curl -X POST https://tu-dominio.com/whatsapp/api/v1/mensajes/enviar/ \
-H "X-API-Key: $NODE_SECRET_KEY" \
-H "Content-Type: application/json" \
-d '{"sesion_id": 1, "destino": "593999999999", "texto": "Hola desde API"}'
```

Webhooks salientes (estilo Zapier)

Qué resuelve: notificar a sistemas externos (Zapier, Make, n8n, HubSpot, tu propio ERP) cuando pasa algo en el CRM.

Setup: admin → **Webhooks salientes** → crear registro con: - **url** destino. - **eventos** suscritos (lista JSON).
Ejemplo: ["conversacion.nueva", "mensaje.entrante", "conversacion.etapa"]. - **secret** (opcional, para

firmar HMAC en header `X-FC-Signature`).

Eventos disponibles: definidos en `EVENTOS_INTEGRACION`. Los más útiles: - `conversacion.nueva` - `conversacion.cerrada` - `conversacion.etapa` (movimiento en Kanban) - `contacto.nuevo` - `mensaje.entrante` - `campana.completada`

Nota: el disparador queda como tarea aparte (plumbing en los modelos — agregar señal + cola). El modelo y el log de entregas (`EntregaWebhookSaliente`) ya están listos.

Cron jobs

Todos viven en `cron_jobs/`:

Script	Frecuencia sugerida	Qué hace
<code>reconectar_sesiones.py</code>	cada 2 min	Rearma sesiones Baileys caídas
<code>enviar_mensaje_despedida.py</code>	cada 5 min	Cierra conversaciones expiradas
<code>enviar_mensajes_programados.py</code>	cada minuto	Despacha mensajes individuales programados
<code>aprender_conversaciones.py</code>	cada hora	Alimenta el vectorstore con nuevos pares Q&A
<code>ejecutar_campanas.py</code> (nuevo)	cada minuto	Arranca + despacha campañas broadcast

Flujo end-to-end: anuncio IG → venta cerrada → Purchase a Meta Ads

1. **Configuras un ad CTWA** en Meta Ads Manager ("Click to WhatsApp") apuntando a tu número.
2. **Un usuario clic en el ad** y envía su primer mensaje en WhatsApp.
3. **Meta webhooke** a tu `/whatsapp/meta_webhook/` con el mensaje + bloque `referral`.
4. **Django:**
5. Crea `Contacto` y `ConversacionWhatsApp` con `ctwa_clid`, `ad_id`, `campaign_id` rellenados.
6. Si la sesión tiene `auto_asignar_round_robin`, asigna al próximo agente disponible.
7. Si hay `pixel_meta` vinculado, dispara evento **Lead** a Meta CAPI con `ctwa_clid` para atribución inmediata.

8. **Tu agente (o la IA)** responde. La conversación vive normalmente.
9. **El agente arrastra la tarjeta** en el Kanban a la etapa "Cerrado ganado" (`es_ganado=True`). Actualiza `valor_estimado` antes.
10. **Django dispara evento Purchase** a CAPI con `value = card.valor_estimado`.
11. **En Meta Ads Manager**: ahora verás la conversión atribuida al `campaign_id` original. El algoritmo puede optimizar para conversiones (no solo clics).
12. **En el dashboard Analytics** → tabla **ROI por campaña CTWA** verás cuántas conversaciones / leads / clientes vinieron de cada ad y la tasa de conversión.

Tutorial paso a paso (demo)

*No puedo grabar un video desde aquí, pero abajo tienes un **runbook click-por-click** que simula uno. Si lo sigues en orden, en ~20 minutos tienes todo operativo.*

Paso 1 · Preparar etiquetas (2 min)

1. `/whatsapp/etiquetas/` → **Nueva etiqueta**
2. Crea estas 4:
3. `VIP` · color rojo
4. `Lead caliente` · color naranja
5. `No molestar` · color gris
6. `Newsletter OK` · color verde

Paso 2 · Crear el pipeline (3 min)

1. `/whatsapp/pipeline/` → **Nuevo pipeline** → "Ventas"
2. Agrega 5 etapas:
3. "Nuevo" gris 10%
4. "Contactado" azul 30%
5. "Cotizando" amarillo 60%
6. "Ganado" verde 100% (marca `es_ganado=true`)
7. "Perdido" rojo 0% (marca `es_perdido=true`)

Paso 3 · Configurar horarios (2 min)

1. [/whatsapp/horarios/](#) → selecciona tu sesión.
2. Mensaje fuera de horario: "Hola, estamos atendiendo fuera de horario. Te responderemos al abrir a las 9am."
3. Zona horaria: [America/Guayaquil](#)
4. Agrega horario lunes a viernes 09:00 → 18:00.

Paso 4 · Setup pixel CAPI (5 min)

1. En **Meta Business Manager**: crea pixel (si no lo tienes) y genera un **CAPI token**.
2. Django admin → **Pixels Meta (CAPI)** → crea registro:
3. nombre: "Pixel principal"
4. pixel_id: [<el ID>](#)
5. access_token: [<el token>](#)
6. test_event_code: dejarlo vacío (o uno de test si quieres probar sin data real).
7. [/whatsapp/sesiones/](#) → edita tu sesión WhatsApp → campo **Pixel Meta (CAPI)** → selecciona el pixel.

Paso 5 · Crear un ad CTWA (fuera del sistema, en Meta Ads Manager)

1. Objetivo: **Engagement** → **Messaging**.
2. Click Destination: **WhatsApp**.
3. Número: el que está linkeado a tu [phone_number_id](#) en [ConfigMeta](#).
4. Publícalo.

Paso 6 · Probar el flujo

1. Desde **otra cuenta WhatsApp**, toca el anuncio y envía un mensaje.
2. En FastChat verás la conversación nueva con:
3. Un ícono ■ en el Kanban si la abres como tarjeta.
4. [ctwa_clid](#), [ad_id](#), [campaign_id](#) rellenos (confirmable en admin [/admin/whatsapp/conversacionwhatsapp/<id>/](#)).
5. En admin → **Eventos CAPI** deberías ver un evento **Lead** enviado, [exitoso=True](#).
6. En Meta Events Manager → **Test Events** (o el pixel regular) verás el Lead.

Paso 7 · Cerrar una venta

1. En la conversación, agrégala al Kanban (desde la vista de conversaciones o admin).
2. En [/whatsapp/pipeline/](#), arrástrala a **Ganado**.
3. Edita la tarjeta y pon [valor_estimado = 100](#), moneda [USD](#).

4. Al soltarla en Ganado, se dispara **Purchase** a CAPI.
5. En admin → **Eventos CAPI** verás el Purchase con `valor=100`.

Paso 8 · Lanzar una campaña

1. `/whatsapp/campanas/` → **Nueva campaña**
2. Nombre: "Promo abril"
3. Sesión: tu sesión WhatsApp
4. Tipo: texto
5. Mensaje: `Hola {nombre}, tenemos una oferta para ti.`
6. Etiquetas incluir: `VIP`
7. Throttle: 20 msg/min
8. Guardar → queda en borrador → **Enviar ahora**
9. Asegúrate que el cron `ejecutar_campanas.py` esté corriendo (mínimo cada minuto).
10. En ~1 min el estado pasa a **enviando**; cuando no quedan pendientes, **completada**.

Paso 9 · Ver analytics

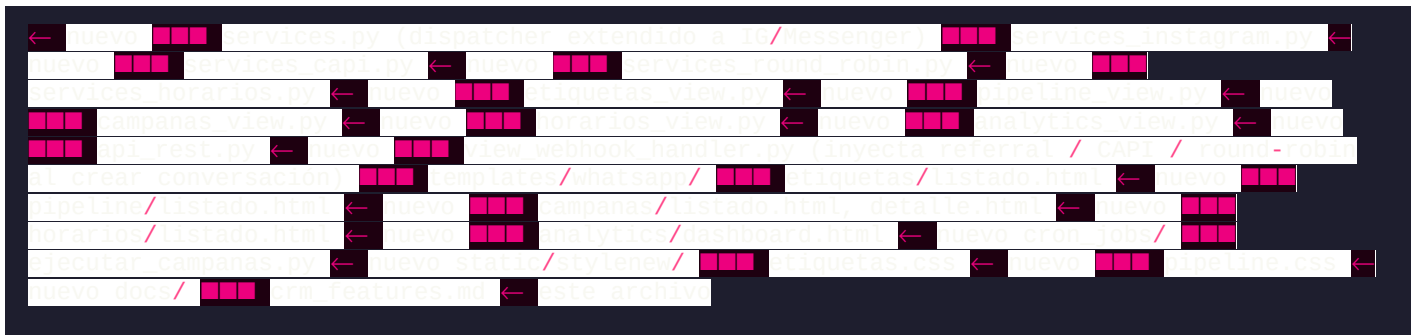
1. `/whatsapp/analytics/`
2. Selecciona rango 30 días.
3. Verás conversaciones por día, atribución CTWA, ranking agentes, forecast pipeline, stats CAPI.

Paso 10 · Integrar con algo externo (Zapier-style)

1. Admin → **Webhooks salientes** → nuevo:
2. url: `https://hooks.zapier.com/hooks/catch/....`
3. eventos: `["conversacion.nueva", "conversacion.etapa"]`
4. secret: genera uno.
5. (Pendiente de cablear los triggers a señales — el modelo y el log ya están listos para cuando lo conectes.)

Estructura de archivos nuevos

<code>models.py</code>	<code>/</code>	<code>models.py</code>	<code>+15</code>	<code>models_nuevos.py</code>	<code>+20</code>	<code>admin.py</code>	<code>(registros para todos los modelos nuevos)</code>	<code>urls.py</code>	<code>(5 vistas + 10 endpoints REST + 2 webhooks sociales)</code>	<code>meta_webhook_view.py</code>	<code>(captura referral CTWA)</code>	<code>meta_social_webhook_view.py</code>	<code>(IG + Messenger)</code>
------------------------	----------------	------------------------	------------------	-------------------------------	------------------	-----------------------	--	----------------------	---	-----------------------------------	--------------------------------------	--	-------------------------------



Roadmap pendiente

- **Multi-tenant** (solicitado explícitamente omitir).
- **Cableado de señales** de modelos → [WebhookSaliente](#) (modelo listo, falta disparador via `post_save` signals).
- **Unificación multi-canal**: [ContactoUnificado](#) para el mismo usuario en IG + WhatsApp + Messenger.
- **Template builder visual** para plantillas Meta (hoy JSON en admin).
- **Integraciones preconfiguradas**: Zapier/Make/HubSpot/Sheets.
- **Transferencia in-call entre agentes** con notas.
- **Exportación CSV/XLSX** desde el listado de contactos y campañas.

Contacto / soporte interno

Documentación de referencia: - [CLAUDE.md](#) (raíz) — visión arquitectural general del proyecto. - [docs/meta_setup.md](#) — setup Meta Cloud API original. - [docs/chatbot_tradicional.md](#) — flujos tradicionales de chatbot.